

Migração gradual para microsserviços

Documentação da transição

Este documento descreve o estado atual do projeto monolítico (conteúdo principal em `src/` e testes em `test/`), aponta pontos positivos e negativos da abordagem atual, explica a separação dos serviços `auth`, `users` e `events` em projetos isolados, um a um e fundamenta os benefícios e riscos da refatoração. O objetivo é servir como material de apoio.

Visão geral do projeto atual

Estrutura principal do repositório (resumo)

src/ código-fonte principal, contendo módulos/serviços:

auth/

Controladores, serviços, DTOs, entidades e estratégias relacionadas à autenticação (JWT, refresh tokens, DTOs de login/logout/refresh)

users/

Endpoints, serviço e entidades de usuário

events/

Lógica e endpoints que manipulam eventos do domínio (DTOs, entidades, serviços)

common/

Guardas, interceptors, validadores, decorators e utilitários compartilhados

database/

Configuração do DataSource, migrations

main.ts, app.module.ts, app.service.ts

Bootstrap e composição do app NestJS

test/ testes e2e, factories e mocks, incluindo `rbac.e2e-spec.ts` e testes para `auth`, `events` e `users`.

Config/infra: Dockerfile, docker-compose.yml, scripts de migração e deploy, package.json, tsconfig*.



Observações rápidas

NestJS

Projeto em **NestJS**
(estrutura modular típica:
módulos, controllers,
services, DTOs, pipes,
guards)

Migrations

Banco de dados gerenciado
via **migrations**
(src/database/migrations)

Testes

Testes e2e e factories já
existem – ponto positivo para
validação após cada migração

Pontos positivos da arquitetura monolítica atual



Simplicidade

Simplicidade ao iniciar e
entender o projeto: um
único repositório, config e
pipeline de CI/CD
simplificam o onboarding



Compartilhamento fácil

Compartilhamento fácil
de código: common/
centralizado evita
duplicação imediata



Coerência transacional

Coerência transacional e
facilidade para migrations:
DataSource único
simplifica evolução do
schema sem coordenação
cross-service



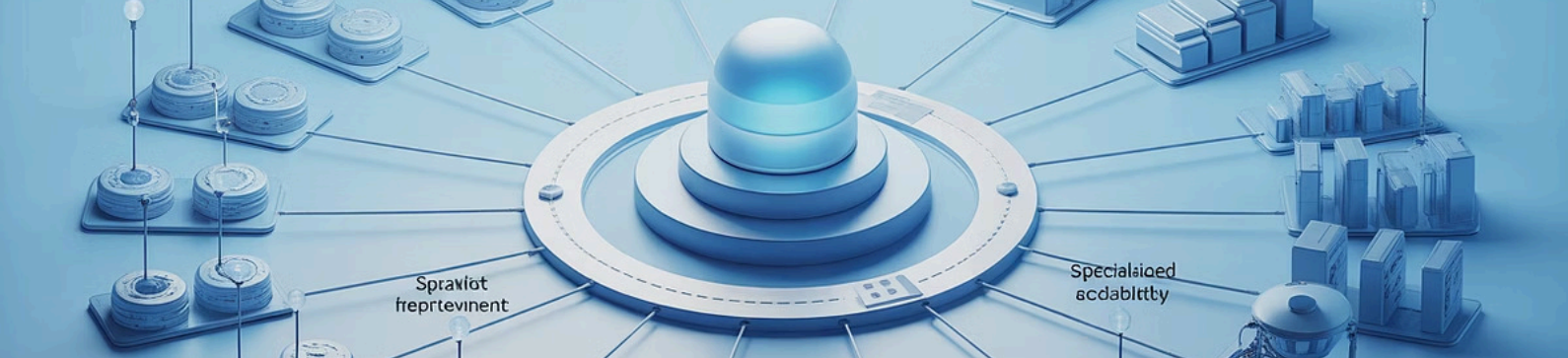
Testes integrados

Testes integrados (e2e) cobrem fluxo
entre módulos



Menor sobrecarga operacional

Menor sobrecarga operacional no início
(um processo, um deploy)



Pontos negativos / limitações do monólito

Por que migrar

1

Escala e autonomia limitadas

Escala e autonomia limitadas: times impactados por deploys cruzados

2

Ciclo de deploy pesado

Ciclo de deploy pesado: pequenas mudanças exigem rebuild/deploy do monólito inteiro

3

Acoplamento tecnológico e de dados

Acoplamento tecnológico e de dados: DB compartilhado e common/ centralizado podem virar gargalo

4

Risco em mudanças de contrato

Risco em mudanças de contrato: sem contratos explícitos, alterações internas quebram dependentes

5

Políticas distintas por serviço

Políticas distintas por serviço são difíceis (escalonamento, linguagem, bancos)

O que faremos agora

Separar serviços em projetos **isolados** de forma **incremental** (auth → users → events).



Fase 1

Levantamento, justificativa, plano e materiais didáticos



Fase 2

Execução da migração passo a passo



Benefícios esperados da refatoração Microserviços



Autonomia

Ciclo de vida independente por serviço



Tecnologia por serviço

Liberdade de stack quando fizer sentido



Deploys menores e frequentes

Redução de risco por release



Contratos explícitos

E versionamento claro (REST/gRPC)



Escalabilidade específica

Por serviço



Foco de equipe

E ensino de **DDD**/bounded contexts

Riscos e custos da migração

Overhead operacional

Mais serviços $\Rightarrow$ mais infra/observabilidade

Complexidade de dados

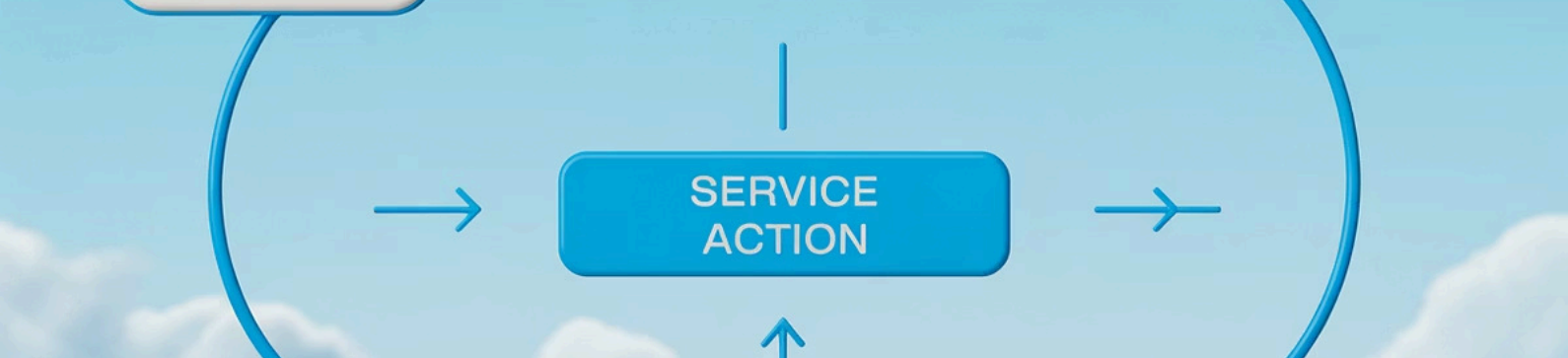
Separar schema exige estratégias (event sourcing, CDC, syncs, domain events)

Latência e resiliência

Chamadas inter-serviço introduzem latência e exigem tolerância a falhas

Esforço de arquitetura

Libs compartilhadas, pipelines CI/CD e catálogo de contratos



Estratégia de migração sugerida

Incremental e repetível

Ordem recomendada: **auth** → **users** → **events** (dependências naturais).

Análise & responsabilidades

Endpoints, DTOs, entidades, dependências, dados compartilhados

Contrato público (API)

Especificação OpenAPI; versão inicial + política de compatibilidade

Extração de código

Criar scaffold (packages/-service/ ou repo separado); mover controllers/services/DTOs/entidades; substituir common/ por lib compartilhada ou duplicação controlada temporária

Infra de dados

Decidir DB/schema próprio vs. shared DB; migrar migrations e bootstrap

AuthN/AuthZ

Padronizar verificação de tokens (JWT/JWKS, introspection, middleware compartilhado)

Comunicação

Começar com **REST** (timeouts/retries); avaliar **eventos** (RabbitMQ/Kafka) para events

Testes & CI

Reaproveitar eze como **testes de contrato**; pipeline CI por serviço (build, lint, unit, eze leves)

Deploy & observabilidade

Dockerizar; health checks; logs estruturados; métricas; tracing

Cutover & monitoramento

Canary/side-by-side; comutar tráfego gradualmente

Limpeza

Remover legado do monólito; common/ vira lib versionada

Contrato mínimo por serviço

Sugestão

Input

Input: HTTP JSON com DTOs validados

Output

Output: JSON + status codes claros; erros { error: string, code?: string }

Erros padrão

- 400 validação
- 401 autenticação
- 403 autorização
- 404 não encontrado
- 500 interno

- ❑ **Critério de sucesso:** endpoints migrados passam **unit + contrato** e comportamento **equivalente** ao monólito em tráfego real.

Casos de borda a considerar

RBAC

Coerência entre serviços; definir **fonte de verdade** (Auth/Identity)

Transações cross-service Sagas/compensações

Dados duplicados

Sincronização e **eventual consistency**

Migração de dados

E **retrocompatibilidade de schema**

crossService

ARCHITECTURE





Material didático e atividades

Tópicos de estudo

- Monólito × Microserviço; **bounded contexts**; trade-offs
- **Contratos**: OpenAPI, versionamento, **testes de contrato**
- **Comunicação**: REST × gRPC × mensagens; **circuit breaker**, retries, idempotência
- **Dados**: padrões de migração, **sagas**, **CDC**
- **Observabilidade & deploy**: logs, métricas, **traces**, health checks

Atividades práticas

1

Tarefa 1

Especificar e documentar API do auth em OpenAPI

2

Tarefa 2

Scaffold de auth-service e mover endpoints de **login/refresh**

3

Tarefa 3

Testes de contrato contra o monólito e validação do comportamento

4

Tarefa 4

Dockerizar e criar pipeline CI básico

PRs **pequenos** e incrementais; relatório de riscos/decisões por serviço; demonstração de **cutover** e **rollback**.



Registro de ações — scaffold do auth-service

Resumo (para criar)

- `packages/auth-service/package.json` scripts (build, start, dev, test, contract:test) e devDeps (TS/Jest)
- `packages/auth-service/tsconfig.json` TS mínimo
- `packages/auth-service/Dockerfile` imagem base
- `packages/auth-service/README.md` guia para alunos (instalar deps, contrato, dev)
- `packages/auth-service/openapi.yaml` contrato com `/auth/login` e `/auth/refresh`
- `packages/auth-service/test/contract/openapi.spec.ts` validação local da OpenAPI
- `packages/auth-service/jest.config.ts` config do Jest

Recomendações práticas (baixo risco) pré-extração

- Adicionar **testes de contrato** (pact/consumer-driven) para integrações
- Versionar **shared-lib** (ou monorepo packages/) para itens realmente compartilhados
- Preparar **infra mínima**: imagens Docker, health checks, jobs CI e (opcional) **broker** de mensagens
- Fazer **backup** das migrations e planejar scripts de migração granular